Efficient Temporal Simple Path Graph Generation

Zhiyang Tang[†], Yanping Wu[‡*], Xiangjun Zai[♮], Chen Chen[§*], Xiaoyang Wang[♮], Ying Zhang[†]

[†]*Zhejiang Gongshang University, China* [‡]*University of Technology Sydney, Australia*
[♮]*The University of New South Wales, Australia* [§]*University of Wollongong, Australia*

zhiyangtang.zjgsu@gmail.com yanping.wu@student.uts.edu.au
xiangjun.zai@student.unsw.edu.au chenc@uow.edu.au
xiaoyang.wang1@unsw.edu.au ying.zhang@zjgsu.edu.cn

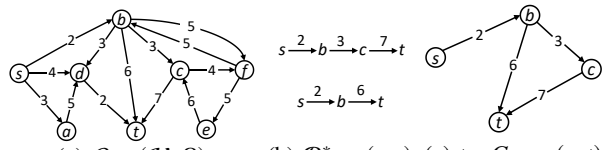
***Abstract*—Interactions between two entities often occur at specific timestamps, which can be modeled as a temporal graph. Exploring the relationships between vertices based on temporal paths is one of the fundamental tasks. In this paper, we conduct the first research to propose and investigate the problem of generating the temporal simple path graph ($tspG$), which is the subgraph consisting of all temporal simple paths from the source vertex to the target vertex within the given time interval. Directly enumerating all temporal simple paths and constructing the $tspG$ is computationally expensive. To accelerate the processing, we propose an efficient method named Verification in Upper-bound Graph. It first incorporates the temporal path constraint and simple path constraint to exclude unpromising edges from the original graph, which obtains a tight upper-bound graph as a high-quality approximation of the $tspG$ in polynomial time. Then, an Escape Edges Verification algorithm is further applied in the upper-bound graph to construct the exact $tspG$ without exhaustively enumerating all temporal simple paths between given vertices. Finally, comprehensive experiments on 10 real-world graphs are conducted to demonstrate the efficiency and effectiveness of the proposed techniques.**

I. Introduction

Path enumeration is a fundamental problem in graph analysis, which aims to list all paths from one vertex to another [1]–[13]. In reality, relationships between entities are often associated with timestamps, which can be modeled as the temporal graph [14]–[27]. Due to the wide applications of temporal graphs in different domains, several path models and enumeration problems have been defined by integrating temporal constraints [12], [13]. For example, infectious disease spread can be analyzed by enumerating temporal paths between infected people based on their movement trajectories [12].

Although path enumeration can benefit many fields, it still may not be adequate for studying relations in certain aspects. Large-scale path enumeration often results in significant overlaps in vertices and edges [1]. Moreover, such an enumeration fails to globally consider and fully utilize the structural connectivity across different paths [28] . To address the challenges, a few works propose the concept of *path graph*, which consists of all paths with certain features between vertices [28]–[30]. For example, [29] defines the k-hop s-t subgraph query for all paths from s to t within k hops, and [30] extends this concept to the hop-constraint simple path graph. Wang et al. [28] define the shortest path graph, which exactly contains all the shortest paths between a pair of vertices. Unfortunately,

*Corresponding author



(a) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (b) $\mathcal{P}^*_{[2,7]}(s,t)$ (c) $tspG_{[2,7]}(s,t)$

Fig. 1: Running example

these path graphs disregard the temporal information, making them applicable only to static networks and unable to capture dynamic features. Furthermore, they impose hop constraint or shortest path constraint, limiting the information they provide.

Motivated by this, in this paper, we propose a novel problem, named temporal simple path graph ($tspG$) generation. It allows for a comprehensive and dynamic analysis of connectivity in temporal graphs, which is crucial for understanding the underlying structure and behavior of real-world systems. In our problem, we focus on the strict and simple temporal path, that is, timestamps of edges along the path follow a strict ascending temporal order and no repeated vertices exist in the path [31]. Specifically, given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, two vertices $s, t \in \mathcal{V}$, and a time interval $[\tau_b, \tau_e]$, the temporal simple path graph, denoted by $tspG_{[\tau_b,\tau_e]}(s,t)$, is the subgraph of $\mathcal{G}$ containing exactly all temporal simple paths from s to t within the time interval $[\tau_b, \tau_e]$.

***Example* 1** *Fig. 1(a) shows a directed temporal graph. Given the source vertex s, the target vertex t and the time interval $[2,7]$, there exist two temporal simple paths from s to t within $[2,7]$, whose details are shown in Fig. 1(b). These two paths share the same edge $e(s,b,2)$, and we illustrate the corresponding temporal simple path graph in Fig. 1(c). As observed, the path graph facilitates path enumeration and explicitly reveals structural connectivity between vertices.*

Applications. In the following, we illustrate four representative applications of temporal simple path graph generation.

- **Outbreak Control.** A disease transmission network can be modeled as a temporal graph, where vertices correspond to locations of individual contact, and temporal edges indicate the movement of individuals to specific locations at particular timestamps [32]. The transmission paths in such a graph reflect the spread of pathogens over time. Generating the temporal simple path graph from the outbreak source to the protected area, health authorities can visualize transmission

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00268

routes, identify critical nodes, and prioritize containment strategies for optimized epidemic control.

- **Financial Monitor.** Temporal graphs can be used to model transaction networks, where each vertex represents an account and each edge represents a transaction from one account to another at a specific time. Money laundering patterns often manifest as cyclic transaction sequences with ascending timestamps within tight temporal windows [33]. A transaction $e(t, s, \tau)$ belongs to such a cycle if a temporal simple path exists from s to t within specified interval [30]. Thus, we can visualize the flow of money from one account to another by our problem, providing a clear representation of the transaction process and aiding in identification of risk-associated accounts and transactions.
- **Travel Planning.** Transportation networks modeled as temporal graphs can efficiently capture dynamic traffic patterns, with vertices denoting intersections or road segments and temporal edges denoting vehicle movements [23], [34]. In a large urban area with a complex public transportation system, passengers often face challenges in planning their routes, especially when they miss their initial connections. By generating the temporal simple path graph, we can visualize all possible transfer options that allow passengers to reach their destinations within a desirable time interval.
- **Trend Detection.** In social networks, information propagation is a time-dependent process. Interactions among users, such as retweets, comments, and likes, can be modeled as temporal graphs [26]. A temporal simple path graph from the information source to a target user within a specified time interval enables the analysis of the information flow and the recognition of key influencers while avoiding the explicit tracing of dissemination routes.

Challenges. To the best of our knowledge, we are the first to propose and investigate the temporal simple path graph generation problem. Naively, we can enumerate all temporal simple paths from the source vertex to the target vertex within the query time interval, then extract distinct vertices and edges from those paths to form the $tspG$. However, this approach is far from efficient since it suffers from an exponential time complexity of $O(d^{\theta} \cdot \theta \cdot m)$, where d is the largest vertex out-degree or in-degree in the original graph $\mathcal{G}$, θ is the length of the query time interval and m is the number of edges in $\mathcal{G}$. The key drawbacks of the naive enumeration strategy can be observed as follows: *i*) The original graph $\mathcal{G}$ includes numerous edges that are definitely not part of any temporal simple paths, which leads to unnecessary large search space for the enumeration. *ii*) Some edges may appear in multiple enumerated temporal simple paths from s to t and will be processed repeatedly during the $tspG$ formation process.

Our solutions. To address the aforementioned drawbacks and efficiently generate the temporal simple path graph, a novel method named *Verification in Upper−bound Graph* (shorted as VUG) is proposed with two main components, i.e., Upper-bound Graph Generation and Escaped Edges Verification.

Specifically, VUG reduces the search space for temporal simple paths by constructing an upper-bound graph as a high-quality approximation of the temporal simple path graph in polynomial time through two phases. In the first phase, it utilizes the QuickUBG method to efficiently filter out edges that are not contained in any temporal paths from s to t within $[\tau_b, \tau_e]$ and obtain a quick upper-bound graph $\mathcal{G}_q$. This can be achieved by comparing the timestamp of an edge with the earliest arrival time $\mathcal{A}(\cdot)$ and latest departure time $\mathcal{D}(\cdot)$ of its end vertices, where $\mathcal{A}(u)$ (resp. $\mathcal{D}(u)$) is when a temporal path from s within $[\tau_b, \tau_e]$ first arrives at u (resp. a temporal path to t within $[\tau_b, \tau_e]$ last departs u). $\mathcal{A}(u)$ and $\mathcal{D}(u)$ for all vertices can be computed using an extended BFS-like procedure in $O(n + m)$ time, where n and m are the number of vertices and edges in $\mathcal{G}$, respectively. In the second phase, VUG applies the TightUBG method on $\mathcal{G}_q$, which further excludes several edges that can be efficiently determined as not contained in any temporal simple path from s to t within $[\tau_b, \tau_e]$. This method is powered by the introduction of *time-stream common vertices*, which summarizes the vertices that commonly appear across all temporal simple paths from s to u (resp. from u to t) within a time interval of interest. The computation of time-stream common vertices utilizes a recursive method in $O(n+\theta \cdot m)$ time without explicitly listing the paths, more clearly, time-stream common vertices of paths from s to vertex u can be calculated from time-stream common vertices of paths from s to the in-neighbors of u. After that, an edge $e(u, v, \tau)$ with two disjoint sets of time-stream common vertices, one set for temporal simple paths from s to u arriving before τ and the other set for temporal simple paths from v to t departuring after τ, is considered to have the potential of forming a temporal simple path from s to t and is retained in the tight upper-bound graph $\mathcal{G}_t$.

Finally, we generate the exact $tspG$ by verifying each edge in $\mathcal{G}_t$. Instead of employing a brute-force path enumeration, we propose an Escaped Edges Verification (EEV) method that mitigates the repetition in edge verification. This method iteratively selects an unverified edge, identifies a temporal simple path from s to t containing it through bidirectional DFS, then adds a set of vertices and edges to the $tspG$. Two carefully designed optimization strategies regarding search direction and neighbor exploration order are integrated into bidirectional DFS to further accelerate the process of algorithm.

Contributions. We summarize the contributions in this paper.

- We conduct the first research to propose and investigate the problem of generating temporal simple path graph.
- To address this challenging problem, we propose an efficient method, namely VUG, consisting of two components, including Upper-bound Graph Generation for effectively yielding a high-quality approximate solution and Escaped Edges Verification for efficiently achieving the exact solution.
- We conduct extensive experiments on 10 real-word temporal graphs to compare VUG against baselines. The results demonstrate the effectiveness and efficiency of our methods.

Note that, due to the limited space, all omitted contents can be found in our appendix online [35].

II. Preliminaries

A directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a set of vertices $\mathcal{V} = \mathcal{V}(\mathcal{G})$ and a set of directed temporal edges $\mathcal{E} = \mathcal{E}(\mathcal{G})$. We use $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$ to denote the number of vertices and edges, respectively. $e(u, v, \tau) \in \mathcal{E}$ denotes a directed temporal edge from vertex u to vertex v, where u, $v \in \mathcal{V}$ and τ is the interaction timestamp from u to v. Without loss of generality, we use the same setting as previous studies for timestamp, which is the integer, since the UNIX timestamps are integers in practice (e.g., [16], [22]). We use $\mathcal{T} = \{\tau | e(u, v, \tau) \in \mathcal{E}\}$ to represent the set of timestamps. Given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a subgraph $\mathcal{S} = (\mathcal{V}_\mathcal{S}, \mathcal{E}_\mathcal{S})$ is an induced subgraph of $\mathcal{G}$, if $\mathcal{E}_\mathcal{S} \subseteq \mathcal{E}$, $\mathcal{V}_\mathcal{S} = \{u | e(u, v, \tau) \in \mathcal{E}_\mathcal{S} \vee e(v, u, \tau) \in \mathcal{E}_\mathcal{S}\}$. Given a subgraph $\mathcal{S}$, let $N_{out}(u, \mathcal{S}) = \{(v, \tau) | e(u, v, \tau) \in \mathcal{E}_\mathcal{S}\}$ (resp. $N_{in}(u, \mathcal{S}) = \{(v, \tau) | e(v, u, \tau) \in \mathcal{E}_\mathcal{S}\}$) be the set of out-neighbors (resp. in-neighbors) of u in $\mathcal{S}$. We use $\mathcal{T}_{out}(u, \mathcal{S})$ (resp. $\mathcal{T}_{in}(u, \mathcal{S})$) to represent all distinct timestamps in $N_{out}(u, \mathcal{S})$ (resp. $N_{in}(u, \mathcal{S})$).

Given a time interval $[\tau_b, \tau_e]$, where τ_b, $\tau_e \in \mathcal{T}$ and $\tau_b \leq \tau_e$, we use θ to denote the span of $[\tau_b, \tau_e]$, i.e., $\theta = \tau_e - \tau_b + 1$. The projected graph of $\mathcal{G}$ within $[\tau_b, \tau_e]$ is a subgraph of $\mathcal{G}$, denoted by $\mathcal{G}_{[\tau_b, \tau_e]} = (\mathcal{V}', \mathcal{E}')$, where $\mathcal{E}' = \{e(u, v, \tau) | e(u, v, \tau) \in \mathcal{E} \wedge \tau \in [\tau_b, \tau_e]\}$ and $\mathcal{V}' = \{u | e(u, v, \tau) \in \mathcal{E}' \vee e(v, u, \tau) \in \mathcal{E}'\}$.

Given two vertices s, $t \in \mathcal{V}$ and a time interval $[\tau_b, \tau_e]$, a temporal path $p_{[\tau_b, \tau_e]}(s, t)$ from s to t in $[\tau_b, \tau_e]$, simplified as p, is a sequence of edges $\langle e(s = v_0, v_1, \tau_1), \ldots, e(v_{l-1}, v_l = t, \tau_l)\rangle$ such that $\tau_b \leq \tau_i < \tau_{i+1} \leq \tau_e$ for all integers $1 \leq i \leq l-1$. $\mathcal{V}(p)$ and $\mathcal{E}(p)$ are the set of vertices and edges included in p, respectively. The length of p is the number of edges on the path, denoted by $l = |\mathcal{E}(p)|$.

***Remark* 1** *Let l be the length of a temporal path $p_{[\tau_b, \tau_e]}(s, t)$, and θ be the span of $[\tau_b, \tau_e]$, it is easy to obtain that $l \leq \theta$.*

***Definition* 1 (Temporal Simple Path)** *Given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, two vertices s, $t \in \mathcal{V}$, and a time interval $[\tau_b, \tau_e]$, a temporal simple path $p^*_{[\tau_b, \tau_e]}(s, t) = \langle e(s = v_0, v_1, \tau_1), \ldots, e(v_{l-1}, v_l = t, \tau_l)\rangle$ is a temporal path from s to t within $[\tau_b, \tau_e]$, without repeated vertices. That is, for any integers $0 \leq i < j \leq l$, $v_i \neq v_j$.*

We use $\mathcal{P}_{[\tau_b, \tau_e]}(s, t)$ to denote the set of all temporal paths within $[\tau_b, \tau_e]$. The vertex set and edge set of $\mathcal{P}_{[\tau_b, \tau_e]}(s, t)$ are denoted by $\mathcal{V}(\mathcal{P}_{[\tau_b, \tau_e]}(s, t)) = \cup_{p \in \mathcal{P}_{[\tau_b, \tau_e]}(s, t)} \mathcal{V}(p)$ and $\mathcal{E}(\mathcal{P}_{[\tau_b, \tau_e]}(s, t)) = \cup_{p \in \mathcal{P}_{[\tau_b, \tau_e]}(s, t)} \mathcal{E}(p)$, respectively. Similarly, we denote the temporal simple path set as $\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t)$, along with its corresponding vertex set $\mathcal{V}(\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t))$ and edge set $\mathcal{E}(\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t))$.

***Definition* 2 (Temporal Simple Path Graph)** *Given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a source vertex $s \in \mathcal{V}$, a target vertex $t \in \mathcal{V}$ and a time interval $[\tau_b, \tau_e]$, the temporal simple path graph from s to t within $[\tau_b, \tau_e]$, denoted by $tspG_{[\tau_b, \tau_e]}(s, t)$, is a subgraph $\mathcal{S} = (\mathcal{V}_\mathcal{S}, \mathcal{E}_\mathcal{S})$ of $\mathcal{G}$ such that $\mathcal{V}_\mathcal{S} = \mathcal{V}(\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t))$ and $\mathcal{E}_\mathcal{S} = \mathcal{E}(\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t))$.*

Problem Statement. Given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a source vertex $s \in \mathcal{V}$, a target vertex $t \in \mathcal{V}$ and a time interval $[\tau_b, \tau_e]$, we aim to efficiently generate the temporal simple path graph $tspG_{[\tau_b, \tau_e]}(s, t)$.

Note that, when the context is clear, we can omit the subscript notation $[\tau_b, \tau_e]$ and vertex pair (s, t), that is, $p_{[\tau_b, \tau_e]}(s, t)$, $p^*_{[\tau_b, \tau_e]}(s, t)$, $\mathcal{P}_{[\tau_b, \tau_e]}(s, t)$, $\mathcal{P}^*_{[\tau_b, \tau_e]}(s, t)$ and $tspG_{[\tau_b, \tau_e]}(s, t)$ can be simplified to p, p^*, $\mathcal{P}$, $\mathcal{P}^*$ and $tspG$.

III. Solution Overview

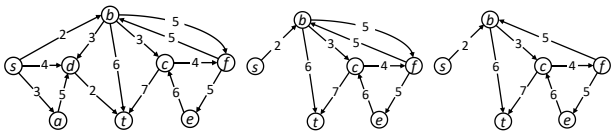
In this section, we first present three reasonable baseline methods to generate temporal simple path graphs by extending the existing studies for temporal simple path enumeration. Then, we introduce the general framework of our solution.

A. The Baseline Methods

A naive method for generating the $tspG_{[\tau_b, \tau_e]}(s, t)$ is to first exhaustively enumerate all temporal simple paths from the source vertex s to the target vertex t within the time interval $[\tau_b, \tau_e]$ in $\mathcal{G}$, then construct the $tspG$ by taking the union of the vertex sets and edge sets of these paths. The running time of this method is bounded by $O(d^\theta \cdot \theta \cdot m)$, where d is the largest out-degree or in-degree of the vertices in $\mathcal{G}$, i.e., $d = \max_{u \in \mathcal{V}}\{\max(|N_{in}(u, \mathcal{G})|, |N_{out}(u, \mathcal{G})|)\}$, and θ is the span of $[\tau_b, \tau_e]$, i.e., $\theta = \tau_e - \tau_b + 1$. Although this approach can return the result, it may not be efficient due to the large search space, which is the original graph, including numerous vertices and edges that definitely cannot be involved in any temporal simple paths. This leads to unnecessary computational costs and slows down the enumeration process.

Baseline methods. To reduce the large search space of the naive method, our baseline methods first filter out many unpromising edges to identify a subgraph of $\mathcal{G}$, which we refer to as the **upper-bound graph** for $tspG$. Then, we enumerate temporal simple paths in the upper-bound graph rather than the original graph, to construct $tspG$.

For the temporal simple path enumeration, a straightforward upper-bound graph would be the projected graph $\mathcal{G}_{[\tau_b, \tau_e]}$ of $\mathcal{G}$, whose computation method is called `dtTSG` in this paper, i.e., it prunes edges $e(u, v, \tau)$ where $\tau \notin [\tau_b, \tau_e]$ in $O(m)$ time. Recently, Jin et al. [12] propose two graph reduction methods (named `esTSG` and `tgTSG`) for temporal path enumeration, and the two reduced graphs can also serve as upper-bound graphs of our problem. Specifically, `esTSG` excludes edges that do not belong to any path from s to t with non-decreasing timestamps, i.e., it performs bidirectional temporal edge traversal (forward with non-decreasing and backward with non-ascending), uses flags to retain vertices and edges that are bidirectionally marked, achieving $O(m)$ time complexity. `tgTSG` discards edges that are not contained in any path from s to t with ascending timestamps, i.e., it employs bidirectional Dijkstra with strict temporal constraints to identify reachable vertices and edges, with an overall time complexity of $O((n + m) \cdot logn + m)$. Obviously, these two methods can construct smaller upper-bound graphs than the projected graph.



(a) dtTSG (b) esTSG (c) tgTSG

Fig. 2: Upper-bound graphs of baselines within interval $[2, 7]$

In our experiment, we design three baseline algorithms, namely EPdtTSG, EPesTSG and EPtgTSG, based on the above three upper-bound graph generation methods, i.e., we first generate an upper-bound graph using each method, then enumerate temporal simple paths on the generated upper-bound graph by initiating a DFS from s to explore all temporal simple paths to t within $[\tau_b, \tau_e]$. During the traversal, for each processed edge, we check its timestamp to ensure a strict temporal constraint in the formed path. After enumeration, we obtain the $tspG$ by combining all vertices and edges from these paths, where each vertex and edge is added to the $tspG$ once through checking the inserted vertices and edges.

***Example* 2** *Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a). Suppose the source vertex is s, the target vertex is t, and the time interval is $[2, 7]$. In Figs. 2(a)-2(c), we illustrate three upper-bound graphs obtained by three baseline algorithms.*

Limitations of baselines. Although the baseline methods can successfully generate temporal simple path graphs, they still have limitations that present opportunities for improvement.

i) The upper-bound graph could be further reduced. For example, edge $e(e, c, 6)$ will not be included in the resulting $tspG$ as it only appears in a temporal path with a cycle. However, since these upper-bound graphs ignore the constraint of the simple path, such edges are retained in these graphs.

ii) Efficiency limitation exists due to the enumeration process. Since some edges may appear in multiple temporal simple paths, the enumeration may lead to repeated verifications to check whether an edge is already included in the result. Therefore, developing efficient techniques for path enumeration can further enhance the performance of the algorithm.

B. The Verification in Upper-bound Graph Framework

To better approximate $tspG$ by the upper-bound graph and avoid repeatedly checking edges that are already added to the result by previously discovered paths, we propose a novel algorithm named *Verification in Upper−bound Graph* (i.e., VUG) with two main components, namely Upper-bound Graph Generation and Escaped Edges Verification, with framework shown in Algorithm 1.

i) **Upper-bound Graph Generation**. We generate the upper-bound graph for $tspG$ in two phases. In the first phase, the quick upper-bound graph $\mathcal{G}_q$ is computed based on temporal constraints. In the second phase, $\mathcal{G}_q$ is further reduced to the tight upper-bound graph $\mathcal{G}_t$ by applying the simple path constraint (more details can be found in Section IV).

ii) **Escaped Edges Verification**. We obtain exact $tspG$ by extending bidirectional DFS, i.e., select an unverified edge $e(u, v, \tau)$, identify a temporal simple path $p^*_{[\tau_b,\tau_e]}(s, t)$ through $e(u, v, \tau)$ and add a set of vertices and edges to $tspG$ (more details can be found in Section V).

Algorithm 1: Framework of VUG

Input : a directed temporal graph $\mathcal{G}$, a source vertex s, a target vertex t and a query time interval $[\tau_b, \tau_e]$

Output: the temporal simple path graph $tspG_{[\tau_b,\tau_e]}(s, t)$

```
// Upper-bound Graph Generation
G_q ← QuickUBG (G, A, D);        /* Algorithm 2 */
G_t ← TightUBG (G_q, TCV);       /* Algorithm 5 */
// Escaped Edges Verification
tspG ← EEV (s, t, [τ_b, τ_e], G_t);   /* Algorithm 6 */
return tspG;
```

IV. Upper-bound Graph Generation

In this section, we present two upper-bound graph computation methods, named quick upper-bound graph generation (QuickUBG) and tight upper-bound graph generation (TightUBG). Given the potentially large size of the original graph, we start with a fast approach QuickUBG to obtain an upper-bound graph $\mathcal{G}_q$ by considering the temporal constraints of our problem. Then, we apply our stronger method TightUBG to obtain a more precise upper-bound graph $\mathcal{G}_t$.

A. Quick Upper-bound Graph Generation

Given a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a temporal simple path graph query of s to t in the time interval $[\tau_b, \tau_e]$, for an edge to be included in the $tspG$, it must be contained in at least one temporal path in $\mathcal{P}_{[\tau_b,\tau_e]}(s, t)$. On this basis, we have the observation below.

***Observation* 1** *An edge $e(u, v, \tau)$ is contained in a temporal path $p_{[\tau_b,\tau_e]}(s, t) \in \mathcal{P}_{[\tau_b,\tau_e]}(s, t)$ iff any one of the following conditions holds:*

i) *$u \neq s$, $v \neq t$, there exist two temporal paths $p_{[\tau_b,\tau_i]}(s, u)$ and $p_{[\tau_j,\tau_e]}(v, t)$ s.t. $\tau_i < \tau < \tau_j$.*

ii) *$u = s$, $v \neq t$, there exist a temporal path $p_{[\tau_j,\tau_e]}(v, t)$ s.t. $\tau_b \leq \tau < \tau_j$.*

iii) *$u \neq s$, $v = t$, there exist a temporal path $p_{[\tau_b,\tau_i]}(s, u)$ s.t. $\tau_i < \tau \leq \tau_e$.*

iv) *$u = s$, $v = t$, $\tau_b \leq \tau \leq \tau_e$.*

Using the above observation as an edge exclusion condition, we can effectively construct an upper-bound graph for $tspG$, referred to as quick upper-bound graph $\mathcal{G}_q$, that only includes edges that appear in at least one temporal path $p_{[\tau_b,\tau_e]}(s, t)$. Before presenting our detailed strategy, we first formally define the concepts of *arrival time* and *departure time*.

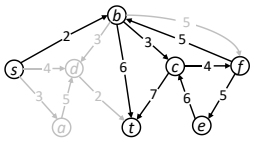
***Definition* 3 (Arrival & Departure Time)** *Given a temporal path $p = p_{[\tau_b,\tau_e]}(s, u) = \langle e(s, \cdot, \cdot), \cdots, e(\cdot, u, \tau_a)\rangle$, the arrival time of u regarding s in this path, denoted by $a(p, u)$, is the timestamp of the last edge within the path, i.e., $a(p, u) = \tau_a$. Similarly, given a temporal path $p = p_{[\tau_b,\tau_e]}(v, t) = \langle e(v, \cdot, \tau_d), \cdots, e(\cdot, t, \cdot)\rangle$, the departure time of v regarding t in this path, denoted by $d(p, v)$, is the timestamp of the first edge within the path, i.e., $d(p, v) = \tau_d$.*

u	$\mathcal{A}(u)$
s	1
a	3
b	2
c	3
d	$4 \rightarrow 3$
e	5
f	$5 \rightarrow 4$
t	$+\infty$

(a) $\mathcal{A}(u)$

u	$\mathcal{D}(u)$
s	$-\infty$
a	$-\infty$
b	6
c	7
d	2
e	6
f	5
t	8

(b) $\mathcal{D}(u)$



(c) $\mathcal{G}_q$

Fig. 3: Quick Upper-bound Graph Generation

Algorithm 2: Quick Upper-bound Graph Generation

Input : a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, earliest arrival time $\mathcal{A}(u)$ and latest departure time $\mathcal{D}(u)$ for each vertex $u \in \mathcal{V}$

Output: quick upper-bound graph $\mathcal{G}_q$=$(\mathcal{V}_q, \mathcal{E}_q)$

```
1 V_q = ∅, E_q = ∅;
2 for each e(u, v, τ) ∈ E do
3     if A(u) < τ ∧ τ < D(v) then
4         V_q ← V_q ∪ {u, v};
5         E_q ← E_q ∪ {e(u, v, τ)};        /* Lemma 1 */
6 return (V_q, E_q);
```

Since each vertex may appear in multiple temporal paths from s to it or from it to t, it can have multiple arrival and departure times. Therefore, we define *polarity time* to store each vertex's earliest arrival time and latest departure time.

***Definition* 4 (Polarity Time)** *For each vertex $u \in \mathcal{V} \setminus \{s\}$, the earliest arrival time of u, denoted by $\mathcal{A}(u)$, is the smallest arrival time of u regarding s among all the paths from s to u within $[\tau_b, \tau_e]$, i.e., $\mathcal{A}(u) = \min\{a(p, u) | p \in \mathcal{P}_{[\tau_b, \tau_e]}(s, u)\}$. Similarly, for each vertex $u \in \mathcal{V} \setminus \{t\}$, the latest departure time of u, denoted by $\mathcal{D}(u)$, is the largest departure time of u regarding t among all the paths from u to t within $[\tau_b, \tau_e]$, i.e., $\mathcal{D}(u) = \max\{d(p, u) | p \in \mathcal{P}_{[\tau_b, \tau_e]}(u, t)\}$.*

***Example* 3** *Given the directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ in Fig. 1(a), a source vertex s, a target vertex t and a time interval $[2, 7]$. For the vertex $f \in \mathcal{V} \setminus \{s, t\}$, $\mathcal{P}_{[2,7]}(s, f) = \{\langle e(s, b, 2), e(b, f, 5)\rangle, \langle e(s, b, 2), e(b, c, 3), e(c, f, 4)\rangle\}$, thus $\mathcal{A}(f) = \min\{4, 5\} = 4$. Similarly, $\mathcal{P}_{[2,7]}(f, t) = \{\langle e(f, e, 5), e(e, c, 6), e(c, t, 7)\rangle, \langle e(f, b, 5), e(b, t, 6)\rangle\}$, thus $\mathcal{D}(f) = 5$.*

With Observation 1 and Definition 4 in hand, we can determine whether an edge is contained in any $p_{[\tau_b, \tau_e]}(s, t)$ based on the polarity time of its end vertices, as stated in Lemma 1. The proof follows directly and is omitted here.

***Lemma* 1** *Given an edge $e(u, v, \tau) \in \mathcal{E}$, there exists a temporal path from the source vertex s to the target vertex t containing $e(u, v, \tau)$ within $[\tau_b, \tau_e]$ iff $\mathcal{A}(u) < \tau < \mathcal{D}(v)$.*

Based on Lemma 1, we can efficiently generate $\mathcal{G}_q$, as illustrated in Algorithm 2. Note that, here we suppose the polarity time of every vertex is already given, and its computation will be discussed later. A running example is provided as follows.

***Example* 4** *Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a), the precomputed polarity time in Figs. 3(a)-3(b), the resulting quick upper-bound graph $\mathcal{G}_q$ is shown in Fig. 3(c). We exclude $e(s, a, 3)$ since $\mathcal{D}(a) = -\infty < 3$, $e(d, t, 2)$ as $\mathcal{A}(d) = 3 > 2$.*

Algorithm 3: Polarity Time Computation

Input : a directed temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a source vertex $s \in \mathcal{V}$, a target vertex $t \in \mathcal{V}$ and a query time interval $[\tau_b, \tau_e]$

Output: the earliest arrival time $\mathcal{A}(u)$ and latest departure time $\mathcal{D}(u)$ for each vertex $u \in \mathcal{V}$

```
1 A(s) ← τ_b − 1, A(u) ← +∞ for each u ∈ V \ {s};
2 D(t) ← τ_e + 1, D(u) ← −∞ for each u ∈ V \ {t};
3 Q ← a queue containing s;
4 while Q ≠ ∅ do
5     u ← Q.pop();
6     for each (v, τ) ∈ N_out(u, G) s.t. v ≠ t do
7         if τ > τ_e ∨ A(u) ≥ τ ∨ τ ≥ A(v) then continue;
8         A(v) ← τ;
9         if τ ≠ τ_e ∧ v ∉ Q then Q.push(v);
10 repeat Lines 3-9 with required adjustment for D(u);
11 return A(u), D(u) for each u ∈ V;
```

***Theorem* 1** *The time complexity of Algorithm 2 is $O(m)$, and its space complexity is $O(n + m)$.*

Due to the space limitation, the proof for Theorem 1 and other omitted proofs can be found in Appendix A online [35].

Next, we introduce our method for calculating the polarity time for each vertex. With a BFS-oriented traversal starting from the source vertex s (resp. the target vertex t), we record and update the earliest arrival time (resp. the latest departure time) for each reachable vertex u when necessary, as there may exist multiple paths from s to u (resp. from u to t). For computational simplicity, we define $\mathcal{A}(s) = \tau_b - 1$, $\mathcal{D}(t) = \tau_e + 1$. The detailed pseudocode of the polarity time computation is given in Algorithm 3. Note that, $\mathcal{A}(u) = +\infty$ (resp. $\mathcal{D}(u) = -\infty$) implies there is no temporal path from s to u (resp. from u to t) within $[\tau_b, \tau_e]$ without passing through t (resp. s).

Polarity Time Computation. In Algorithm 3, $\mathcal{G}$ is stored such that, within the neighbor list $N_{out}(u, \mathcal{G})$, $N_{in}(u, \mathcal{G})$ of each vertex $u \in \mathcal{V}(\mathcal{G})$, neighbors are sorted based on their timestamps. In Lines 4-9, for each popped vertex u from Q, we iteratively explore its out-neighbors. It is worth mentioning that, a pointer in $N_{out}(u, \mathcal{G})$ is maintained for each vertex $u \in \mathcal{V}(\mathcal{G})$ to help filter out (v, τ) that has been processed during a previous visit to u, which avoids scanning the entire $N_{out}(u, \mathcal{G})$ each time u is visited. In Line 7, an out-neighbor (v, τ) with $\mathcal{A}(u) \geq \tau$ or $\tau \geq \mathcal{A}(v)$ is ignored since a new temporal path $p_{[\tau_b, \tau]}(s, v)$ cannot be formed or $p_{[\tau_b, \tau]}(s, v)$ has already been explored previously. We update $\mathcal{A}(v)$ in Line 8 as we find a new temporal path $p_{[\tau_b, \tau]}(s, v)$ with an earlier arrival time for v, and in Line 9 we add v to Q for further exploration if it is not yet in Q. We also repeat the process in Lines 3-9 by toggling between s and t, "out" and "in", $\mathcal{A}$ and $\mathcal{D}$, τ_e and τ_b, $>$ and $<$, $\geq$ and $\leq$ to obtain $\mathcal{D}(\cdot)$.

***Example* 5** *Given the directed temporal graph $\mathcal{G}$ in Fig. 1(a). For the query of s to t within $[2, 7]$, the earliest arrival time*

of each vertex is shown in Fig. 3(a). As we process s, its out-neighbors $\{(b,2),(a,3),(d,4)\}$ *are checked. We add* b, a, d *to* Q *and update their earliest arrival time to* $2, 3, 4$*, respectively. Next,* $\mathcal{A}(d)$ *is updated to* 3 *when* b *is being processed. Note that, we do not add* d *to* Q *since it is currently in* Q*. Then,* a *is processed without changes to* $\mathcal{A}(d)$*, since* $5 > 3 = \mathcal{A}(d)$*.*

***Theorem* 2** *The time complexity of Algorithm 3 is* $O(n+m)$*, and its space complexity is* $O(n)$*.*

Discussions between `tgTSG` and `QuickUBG`. Note that, although both `tgTSG` and `QuickUBG` adopt a strict temporal path constraint and achieve the same effect in terms of graph reduction, `QuickUBG` reduces the time complexity of `tgTSG` by a factor of $O(\log n)$. This is because, `QuickUBG` iteratively updates the polarity time for each vertex, avoiding the overhead of a priority queue used in the Dijkstra-based method `tgTSG`. Experiments also show that `QuickUBG` achieves a speedup of two orders of magnitude compared to `tgTSG` (more details can be found in Section VI).

B. Tight Upper-bound Graph Generation

In this subsection, we present the tight upper-bound graph $\mathcal{G}_t$, which further shrink the quick upper-bound graph $\mathcal{G}_q$ based on the simple path constraint. Our method employs a necessary but not sufficient condition for the inclusion of an edge in the $tspG$. Before introducing the details of this condition, we first give the analysis in the following. Given the quick upper-bound graph $\mathcal{G}_q$ of a directed temporal graph $\mathcal{G}$ for the query of s to t within $[\tau_b, \tau_e]$, following Algorithms 2 and 3, it is straightforward to obtain Lemma 2 below.

***Lemma* 2** *If an edge* $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$*, then one of the following two conditions must be satisfied:*

i) $u \neq s$*,* $v \neq t$*, there exist two temporal simple paths* $p^*_{[\tau_b,\tau_i]}(s,u)$ *and* $p^*_{[\tau_j,\tau_e]}(v,t)$ *s.t.* $\tau_i < \tau < \tau_j$*,* $t \notin p^*_{[\tau_b,\tau_i]}(s,u)$*,* $s \notin p^*_{[\tau_j,\tau_e]}(v,t)$*.*

ii) $u = s$ *or* $v = t$*, there exists a temporal simple path* $p^*_{[\tau_b,\tau_e]}(s,t)$ *s.t.* $e(u,v,\tau) \in p^*_{[\tau_b,\tau_e]}(s,t)$*.*

According to Lemma 2, given an edge $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$, with $u \neq s$ and $v \neq t$, we can always find a pair of simple paths $p^*_{[\tau_b,\tau_i]}(s,u)$ and $p^*_{[\tau_j,\tau_e]}(v,t)$. However, those two paths may share the same vertex which prevents the formation of a simple path $p^*_{[\tau_b,\tau_e]}(s,t)$, as shown in the observation below.

***Observation* 2** *An edge* $e(u,v,\tau)$*, with* $u \neq s$ *and* $v \neq t$*, belongs to a temporal simple path* $p^*_{[\tau_b,\tau_e]}(s,t)$ *iff there exist two temporal simple paths* $p^*_{[\tau_b,\tau_i]}(s,u)$ *and* $p^*_{[\tau_j,\tau_e]}(v,t)$ *s.t. i)* $\tau_i < \tau < \tau_j$ *and ii)* $\mathcal{V}(p^*_{[\tau_b,\tau_i]}(s,u)) \cap \mathcal{V}(p^*_{[\tau_j,\tau_e]}(v,t)) = \emptyset$*.*

Following Observation 2, to determine whether $e(u,v,\tau)$ is contained in any $p^*_{[\tau_b,\tau_e]}(s,t)$, we would need to exhaustively evaluate condition *ii)* for each combination of $p^*_{[\tau_b,\tau_i]}(s,u)$ and $p^*_{[\tau_j,\tau_e]}(v,t)$, which is computationally expensive. Thus, we aim to efficiently identify and remove edges that clearly violate condition *ii)*, rather than all such edges. The high-level idea is that, if there exists a vertex w appearing in all $p^*_{[\tau_b,\tau_i]}(s,u)$ and $p^*_{[\tau_j,\tau_e]}(v,t)$, condition *ii)* will not hold. Therefore, we will elaborate on identifying such common vertex w ($w \neq s$ and $w \neq t$ as discussed in Lemma 2), and formally define the *time-stream common vertices* as follows.

***Definition* 5 (Time-stream Common Vertices)** *Given* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$*, two vertices* $s, t \in \mathcal{V}$*, a time interval* $[\tau_b, \tau_e]$ *and a timestamp* $\tau \in [\tau_b, \tau_e]$*, the time-stream common vertices of* $\mathcal{P}^*_{[\tau_b,\tau]}(s,u)$*, denoted by* $TCV_\tau(s,u)$*, is the set of vertices (except* s*) appearing in all temporal simple paths from* s *to* u *not containing* t *within time interval* $[\tau_b, \tau]$*, i.e.,*

$$TCV_\tau(s,u) = \bigcap_{p^* \in \mathcal{P}^*_{[\tau_b,\tau]}(s,u) \ s.t. \ t \notin \mathcal{V}(p^*)} \mathcal{V}(p^*) \setminus \{s\} \quad (1)$$

Similarly, given $\tau \in [\tau_b, \tau_e]$*,* $TCV_\tau(u,t)$ *is the set of vertices (except* t*) appearing in all* $p^*_{[\tau,\tau_e]}(u,t)$ *not containing* s*, i.e.,*

$$TCV_\tau(u,t) = \bigcap_{p^* \in \mathcal{P}^*_{[\tau,\tau_e]}(u,t) \ s.t. \ s \notin \mathcal{V}(p^*)} \mathcal{V}(p^*) \setminus \{t\} \quad (2)$$

***Lemma* 3** *Given an edge* $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$ *with* $u \neq s$*,* $v \neq t$*, if it is contained in a temporal simple path* $p^*_{[\tau_b,\tau_e]}(s,t)$*, then* $\exists \tau_i, \tau_j \in [\tau_b, \tau_e]$ *s.t. i)* $\tau_i < \tau < \tau_j$*, and ii)* $TCV_{\tau_i}(s,u) \cap TCV_{\tau_j}(v,t) = \emptyset$*; but not necessarily the reverse.*

Lemma 3 reveals that, for the edge $e(u,v,\tau)$, if it satisfies the condition that $\exists\ \tau_i, \tau_j \in [\tau_b, \tau_e]$ s.t. $\tau_i < \tau < \tau_j$, $TCV_{\tau_i}(s,u) \cap TCV_{\tau_j}(v,t) = \emptyset$, then it has the potential to form a $p^*_{[\tau_b,\tau_e]}(s,t)$, and we include it in $\mathcal{G}_t$ for further verification; if not, we can directly conclude this edge is unpromising and exclude it from $\mathcal{G}_t$. However, it is non-trivial to efficiently compute and store time-stream common vertices $TCV_\tau(s,u)$ and $TCV_\tau(u,t)$ for all vertices $u \in \mathcal{V}(\mathcal{G}_q)$ and all timestamps $\tau \in [\tau_b, \tau_e]$, with the main challenges being two-fold.

i) Storing time-stream common vertices for all $\tau \in [\tau_b, \tau_e]$ requires significant space when the span of $[\tau_b, \tau_e]$ is large.

ii) Explicitly enumerating temporal simple paths to obtain time-stream common vertices is computation-intensive.

To address the first challenge, for each vertex $u \in \mathcal{V}(\mathcal{G}_q) \setminus \{s,t\}$, instead of computing and storing time-stream common vertices $TCV_\tau(s,u)$ (resp. $TCV_\tau(u,t)$) for every possible timestamp $\tau \in [\tau_b, \tau_e]$, we can compute and store those for only a finite subset of timestamps. Time-stream common vertices for other timestamps can be inferred from stored information. This idea is supported by the following lemmas.

***Lemma* 4** *For each in-coming edge* $e(v,u,\tau)$ *(resp. out-going edge* $e(u,v,\tau)$*) of* u *in* $\mathcal{G}_q$*, there exists a temporal simple path* $p^*_{[\tau_b,\tau]}(s,u)$ *(resp.* $p^*_{[\tau,\tau_e]}(u,t)$*) such that* $e(v,u,\tau) \in \mathcal{E}(p^*_{[\tau_b,\tau]}(s,u))$ *(resp.* $e(u,v,\tau) \in \mathcal{E}(p^*_{[\tau,\tau_e]}(u,t))$*) and* $t \notin \mathcal{V}(p^*_{[\tau_b,\tau]}(s,u))$ *(resp.* $s \notin \mathcal{V}(p^*_{[\tau,\tau_e]}(u,t))$*).*

***Lemma* 5** *Given* τ *and* $\tau_l = \max\{\tau_i | \tau_i \in \mathcal{T}_{in}(u,\mathcal{G}_q), \tau_i \leq \tau\}$*, we have* $TCV_\tau(s,u) = TCV_{\tau_l}(s,u)$*. Similarly, given* $\tau_r = \min\{\tau_j | \tau_j \in \mathcal{T}_{out}(u,\mathcal{G}_q), \tau_j \geq \tau\}$*,* $TCV_\tau(u,t) = TCV_{\tau_r}(u,t)$*.*

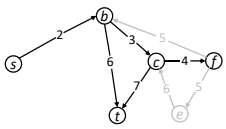
Recall that, $\mathcal{T}_{in}(u,\mathcal{G}_q)$ (resp. $\mathcal{T}_{out}(u,\mathcal{G}_q)$) represents all distinct timestamps in $N_{in}(u,\mathcal{G}_q)$ (resp. $N_{out}(u,\mathcal{G}_q)$). We only

u	τ	$TCV_\tau(s,u)$
b	2	$\{b\}$
	5	*null*
c	3	$\{b,c\}$
	6	$\{b,c\}$
f	4	$\{b,c,f\}$
e	5	$\{b,c,f,e\}$

(a) $TCV_\tau(s,u)$

u	τ	$TCV_\tau(u,t)$
b	6	$\{b\}$
	3	*null*
c	7	$\{c\}$
	4	*null*
f	5	$\{c,e,f\} \rightarrow \{f\}$
e	6	$\{c,e\}$

(b) $TCV_\tau(u,t)$



(c) $\mathcal{G}_t$

Fig. 4: Tight Upper-bound Graph Generation

need to compute and store $TCV_\tau(s,u)$ (resp. $TCV_\tau(u,t)$) for every timestamp in $\mathcal{T}_{in}(u,\mathcal{G}_q)$ (resp. $\mathcal{T}_{out}(u,\mathcal{G}_q)$) based on Lemmas 4 and 5, and then organize them in ascending (resp. descending) order based on τ. Note that, we refer to the stored time-stream common vertices for a timestamp as an *entry*.

***Example* 6** *Following Example 4 and given the quick upper-bound graph $\mathcal{G}_q$ shown in Fig. 3(c), Figs. 4(a)-4(b) show the stored $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$ for all vertices in $\mathcal{G}_q$. Take $TCV.(f,t)$ as an example, $N_{out}(f,\mathcal{G}_q) = \{(b,5),(e,5)\}$, therefore $\mathcal{T}_{out}(f,\mathcal{G}_q) = \{5\}$. As depicted in Fig. 4(b), there is only one entry in $TCV.(f,t)$, which is $TCV_5(f,t)$.*

Regarding the second challenge, according to Lemma 4, a temporal simple path $p^*_{[\tau_b,\tau]}(s,u)$ is composed of two parts, $\langle e(v,u,\tau')\rangle$ and $p^*_{[\tau_b,\tau'-1]}(s,v)$, where $(v,\tau') \in N_{in}(u,\mathcal{G}_q)$, $\tau' \leq \tau$ and $u \notin \mathcal{V}(p^*_{[\tau_b,\tau'-1]}(s,v))$, this inspires us to compute time-stream common vertices of a vertex based on those of its neighbors. However, the main obstacle is the need to check for the simple path constraint during the computation. To overcome this obstacle, we propose the following lemma.

***Lemma* 6** *Intersecting the vertex set of each temporal simple path is equivalent to intersecting that of each temporal path, i.e., $TCV_\tau(s,u) = \bigcap_{p\in\mathcal{P}_{[\tau_b,\tau]}(s,u)\ s.t.\ t\notin\mathcal{V}(p)} \mathcal{V}(p)\setminus\{s\}$; similarly, $TCV_\tau(u,t) = \bigcap_{p\in\mathcal{P}_{[\tau,\tau_e]}(u,t)\ s.t.\ s\notin\mathcal{V}(p)} \mathcal{V}(p)\setminus\{t\}$.*

According to Lemma 6, we can compute $TCV_\tau(s,u)$ (resp. $TCV_\tau(u,t)$) based on the temporal paths $\mathcal{P}_{[\tau_b,\tau]}(s,u)$ (resp. $\mathcal{P}_{[\tau,\tau_e]}(u,t)$) without considering the simple path constraint. Given a vertex $w \neq s$, $w \in TCV_\tau(s,u)$ iff $\forall\ (v,\tau') \in N_{in}(u,\mathcal{G}_q)$ s.t. $\tau' \leq \tau$ and $w \in (TCV_{\tau'-1}(s,v) \cup \{u\})$. Therefore, intersecting $(TCV_{\tau'-1}(s,v)\cup\{u\})$ for each $(v,\tau') \in N_{in}(u,\mathcal{G}_q)$ s.t. $\tau' \leq \tau$ leads to $TCV_\tau(s,u)$. Based on these, we design the computation of $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$ in a recursive way with the formulas formally defined below.

Recursive Computation. Given the source vertex and target vertex $s,t \in \mathcal{V}(\mathcal{G}_q)$, the time interval $[\tau_b,\tau_e]$, for each vertex $u \in \mathcal{V}(\mathcal{G}_q)\setminus\{s,t\}$ and timestamp $\tau \in [\tau_b,\tau_e]$, $TCV_\tau(s,u)$ is computed based on the time-stream common vertices of u's in-neighbors. For ease of computation, we define the base case to be: for $\forall\tau \in [\tau_b-1,\tau_e-1]$, $TCV_\tau(s,s) = \emptyset$.

$$TCV_\tau(s,u) = \bigcap_{\substack{(v,\tau')\in N_{in}(u,\mathcal{G}_q)\\ \tau'\leq\tau}} (TCV_{\tau'-1}(s,v)\cup\{u\}) \tag{3}$$

Similarly, $TCV_\tau(u,t)$ is computed based on the time-stream common vertices of u's out-neighbors. And we define the base case to be: for $\forall\tau \in [\tau_b+1,\tau_e+1]$, $TCV_\tau(t,t) = \emptyset$.

Algorithm 4: Time-stream Common Vertices Computation

Input : quick upper-bound graph $\mathcal{G}_q$ of a temporal graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$, a source vertex $s \in \mathcal{V}$, a target vertex $t \in \mathcal{V}$ and a time interval $[\tau_b,\tau_e]$

Output: the time-stream common vertices $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$

1 $TCV_{\tau_b-1}(s,s) \leftarrow \emptyset$, $P(s) \leftarrow 1$;
2 $TCV_{\tau_e+1}(t,t) \leftarrow \emptyset$, $P(t) \leftarrow 1$;
3 **for** $u \in \mathcal{V}(\mathcal{G}_q)\setminus\{s,t\}$ **do**
4 $\quad TCV_\tau(s,u) \leftarrow null$ for each $\tau \in \mathcal{T}_{in}(u,\mathcal{G}_q)$;
5 $\quad$ mark u as uncompleted;
6 $\quad P(u) \leftarrow 1$;
7 **for** $e(v,u,\tau) \in \mathcal{E}(\mathcal{G}_q)$ **do**
8 $\quad$ **if** $u = t \vee u$ *is completed* **then continue**;
9 $\quad i \leftarrow P(u)$, $\tau_i \leftarrow i$-th timestamp in $TCV.(s,u)$;
10 $\quad TCV_{\tau_i}(s,u) \leftarrow i$-th entry in $TCV.(s,u)$;
11 $\quad j \leftarrow P(v)$, $\tau_j \leftarrow j$-th timestamp in $TCV.(s,v)$;
12 $\quad$ **if** $\tau_j = \tau$ **then**
13 $\quad\quad j \leftarrow j-1$, $\tau_j \leftarrow j$-th timestamp in $TCV.(s,v)$;
14 $\quad TCV_{\tau_j}(s,v) \leftarrow j$-th entry in $TCV.(s,v)$;
15 $\quad$ **if** $TCV_{\tau_j}(s,v) = null$ **then** $TCV_{\tau_j}(s,v) \leftarrow \{v\}$;
16 $\quad$ **if** $TCV_{\tau_i}(s,u) = null$ **then**
17 $\quad\quad TCV_\tau(s,u) \leftarrow TCV_{\tau_j}(s,v) \cup \{u\}$;
18 $\quad$ **else**
19 $\quad\quad TCV_\tau(s,u) \leftarrow TCV_{\tau_i}(s,u) \cap (TCV_{\tau_j}(s,v) \cup \{u\})$;
20 $\quad\quad$ **if** $\tau > \tau_i$ **then** $i \leftarrow i+1$, $P(u) \leftarrow i$;
21 $\quad i$-th entry in $TCV.(s,u) \leftarrow TCV_\tau(s,u)$;
22 $\quad$ **if** $TCV_\tau(s,u) = \{u\}$ **then**
23 $\quad\quad$ mark u as completed; /* Lemma 7 */
24 repeat Lines 3-23 with required adjustment for $TCV.(\cdot,t)$;
25 **return** $TCV.(s,\cdot)$, $TCV.(\cdot,t)$;

$$TCV_\tau(u,t) = \bigcap_{\substack{(v,\tau')\in N_{out}(u,\mathcal{G}_q)\\ \tau'\geq\tau}} (TCV_{\tau'+1}(v,t)\cup\{u\}) \tag{4}$$

The computation of $TCV_\tau(s,u)$ can be interpreted as two steps: *i*) intersect $(TCV_{\tau'-1}(s,v) \cup \{u\})$ for all $(v,\tau') \in N_{in}(u,\mathcal{G}_q)$ s.t. $\tau' \leq \tau-1$; *ii*) intersect $(TCV_{\tau-1}(s,v)\cup\{u\})$ for all $(v,\tau) \in N_{in}(u,\mathcal{G}_q)$. The computation of $TCV_\tau(u,t)$ can be interpreted in a similar manner, and we omit the details here. Clearly, the result of step *i*) is $TCV_{\tau-1}(s,u)$ if $TCV_{\tau-1}(s,u)$ exists, therefore, $TCV_\tau(s,u) \subseteq TCV_{\tau-1}(s,u)$, which allows the computation of $TCV_\tau(s,u)$ to inherit from $TCV_{\tau-1}(s,u)$ and we also have the following lemma that serves as a pruning rule in the computation of time-stream common vertices.

***Lemma* 7** *If $TCV_{\tau_m}(s,u) = \{u\}$, then $\forall\ \tau_n > \tau_m$, $TCV_{\tau_n}(s,u) = \{u\}$. Similarly, if $TCV_{\tau_m}(u,t) = \{u\}$, then $\forall\ \tau_n < \tau_m$, $TCV_{\tau_n}(u,t) = \{u\}$.*

Time-stream Common Vertices Computation. The detailed pseudocode is shown in Algorithm 4. Lines 3-23 compute $TCV_\tau(s,u)$ for each vertex $u \in \mathcal{V}(\mathcal{G}_q)\setminus\{s,t\}$ where τ

$\in \mathcal{T}_{in}(u, \mathcal{G}_q)$. All entries in $TCV.(s,u)$ are initially set to *null* in Line 4. A pointer $P(u)$ in $TCV.(s,u)$ is maintained for each vertex u to enable constant-time access to the entry currently being processed. $P(u)$ is initialized to 1 in Line 6, indicating the $first$ entry (alternatively, the $first$ timestamp) in $TCV.(s,u)$ is initially in processing.

Edges in $\mathcal{E}(\mathcal{G}_q)$ are arranged in non-descending temporal order when $\mathcal{G}_q$ is generated, and Line 7 scans them forward to compute $TCV.(s,\cdot)$. For each scanned edge $e(v,u,\tau)$, we process $TCV_\tau(s,u)$ accordingly, its iterative intersection computation will be completed when all in-coming edges of u with timestamps no greater than τ have been scanned. This ensures the processing of $TCV.(s,\cdot)$ with a smaller timestamp is finalized before the processing of $TCV.(s,\cdot)$ with a larger timestamp begins. For a scanned edge $e(v,u,\tau)$, Lines 11-13 obtain τ_j as either the currently processed timestamp or the immediately preceding timestamp in $TCV.(s,v)$, whichever is closer to but smaller than τ. Then, $TCV_{\tau_j}(s,v)$ is accessed in Lines 14-15 through the pointer corresponding to τ_j in $TCV.(s,v)$, and $TCV_{\tau_j}(s,v)$ is equal to $TCV_{\tau-1}(s,v)$ according to Lemma 5. $TCV_{\tau_i}(s,u) = null$ in Line 16 suggests we are at the beginning of the process for the $first$ entry in $TCV.(s,u)$ and we compute this entry for the first time in Line 17. If $TCV_{\tau_i}(s,u) \neq null$, we are facing two circumstances: *i*) when $\tau = \tau_i$, indicating the i-th entry in $TCV.(s,u)$ has been computed using other in-coming edges of u with the same timestamp τ, and we continue the iterative intersection process for the i-th entry; *ii*) when $\tau > \tau_i$, it indicates that all in-coming edges of u with the timestamp τ_i has been scanned, we can conclude the i-th entry in $TCV.(s,u)$ is finalized and start the processing of the $(i+1)$-th entry. In both cases, Line 19 fulfills the purpose, and $TCV_\tau(s,u)$ is efficiently stored at its position following the pointer $P(u)$ in Line 21.

Lines 22-23 and Line 8 implement the pruning strategy based on Lemma 7, specifically, if $TCV_\tau(s,u) = \{u\}$, any entry in $TCV.(s,u)$ with a larger timestamp than τ will be $\{u\}$, thus, we can omit the further computation of $TCV.(s,u)$ and mark u as completed. As a result, in Line 15, since $\tau_j < \tau$, $TCV_{\tau_j}(s,v)$ should already been finalized and $TCV_{\tau_j}(s,v) = null$ if and only if v was completed previously, therefore we can conclude $TCV_{\tau_j}(s,v) = \{v\}$.

We compute $TCV.(\cdot,t)$ by repeating the process in Lines 3-23 but scanning edges backward in Line 7 and toggling between s and t, "in" and "out".

***Example* 7** *Consider $\mathcal{G}_q$ in Fig. 3(c), the time-stream common vertices $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$ are shown in Figs. 4(a)-4(b). Take the computation of $TCV_5(f,t)$ as an example. When $e(f,e,5)$ is scanned, we have $i = 1$, $\tau_i = 5$, $\tau_j = 6$. Since $TCV_5(f,t) = null$, we are starting to process the first entry in $TCV.(f,t)$ and we obtain $TCV_5(f,t) = TCV_6(e,t) \cup \{f\} = \{c,e\} \cup \{f\} = \{c,e,f\}$ in Line 17. After that, we process $e(f,b,5)$ with $i = 1$, $\tau_i = 5$, $\tau_j = 6$. Since $TCV_5(f,t) \neq null$ and $\tau = 5 = \tau_i$, we continue the iterative intersection process for the first entry in $TCV_5(\cdot,t)$ and update $TCV_5(f,t)$ to $\{c,e,f\} \cap (TCV_6(b,t) \cup \{f\}) = \{f\}$. Then, the pruning condition in Line 22 is satisfied, and we mark f as completed.*

Algorithm 5: Tight Upper-bound Graph Generation

Input : quick upper-bound graph $\mathcal{G}_q$=$(\mathcal{V}_q, \mathcal{E}_q)$, the time-stream common vertices $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$

Output: tight upper-bound graph $\mathcal{G}_t$=$(\mathcal{V}_t, \mathcal{E}_t)$

1 $\mathcal{V}_t = \emptyset, \mathcal{E}_t = \emptyset$;
2 $P_s(u) \leftarrow 1$, $P_t(u) \leftarrow |\mathcal{T}_{out}(u, \mathcal{G}_q)|$ **for each** $u \in \mathcal{V}_q$;
3 **for** $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$ **do**
4 **if** $u = s$ *or* $v = t$ **then**
5 $\mathcal{V}_t \leftarrow \mathcal{V}_t \cup \{u,v\}$, $\mathcal{E}_t \leftarrow \mathcal{E}_t \cup \{e(u,v,\tau)\}$;
6 **continue**; /* Lemma 2 */
7 $i \leftarrow P_s(u)$, $j \leftarrow P_t(v)$;
8 **while** i-th timestamp in $TCV.(s,u) < \tau$ **do** $i \leftarrow i+1$;
9 $i \leftarrow i-1$, $\tau_i \leftarrow i$-th timestamp in $TCV.(s,u)$;
10 **while** j-th timestamp in $TCV.(v,t) \leq \tau$ **do** $j \leftarrow j-1$;
11 $\tau_j \leftarrow j$-th timestamp in $TCV.(v,t)$;
12 $P_s(u) \leftarrow i$, $P_t(v) \leftarrow j$;
13 $TCV_{\tau_i}(s,u) \leftarrow i$-th entry in $TCV.(s,u)$;
14 **if** $TCV_{\tau_i}(s,u) = null$ **then** $TCV_{\tau_i}(s,u) = \{u\}$;
15 $TCV_{\tau_j}(v,t) \leftarrow j$-th entry in $TCV.(v,t)$;
16 **if** $TCV_{\tau_j}(v,t) = null$ **then** $TCV_{\tau_j}(v,t) = \{v\}$;
17 **if** $TCV_{\tau_i}(s,u) \cap TCV_{\tau_j}(v,t) = \emptyset$ **then**
18 $\mathcal{V}_t \leftarrow \mathcal{V}_t \cup \{u,v\}$;
19 $\mathcal{E}_t \leftarrow \mathcal{E}_t \cup \{e(u,v,\tau)\}$; /* Lemma 3 */
20 **return** $(\mathcal{V}_t, \mathcal{E}_t)$;

***Theorem* 3** *The time complexity of Algorithm 4 is $O(n+\theta\cdot m)$ where θ is the span of $[\tau_b, \tau_e]$, i.e., $\theta = \tau_e - \tau_b + 1$. The space complexity of Algorithm 4 is $O(n+\theta\cdot m)$.*

With the precomputed time-stream common vertices, we can exclude unpromising edges from $\mathcal{G}_q$ to generate $\mathcal{G}_t$ following the contrapositive of Lemma 3, i.e., for each $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$, check whether $\forall\ \tau_i, \tau_j \in [\tau_b, \tau_e]$ s.t. $\tau_i < \tau < \tau_j$, $TCV_{\tau_i}(s,u) \cap TCV_{\tau_j}(v,t) \neq \emptyset$. During the process, the following two optimization techniques can be used to accelerate computations. First, based on Lemma 2, all the out-going edges from s and in-coming edges to t can be directly added to $\mathcal{G}_t$. Second, instead of exhausting all possible combinations of τ_i and τ_j and intersecting the corresponding time-stream common vertices, we focus on the maximum timestamp τ_i in $\mathcal{T}_{in}(u, \mathcal{G}_q)$ and minimum timestamp τ_j in $\mathcal{T}_{out}(v, \mathcal{G}_q)$.

***Lemma* 8** *Given an edge $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$, $u \neq s$, $v \neq t$, let $\tau_l = \max\{\tau_i | \tau_i \in \mathcal{T}_{in}(u, \mathcal{G}_q) \wedge \tau_b \leq \tau_i < \tau\}$ and $\tau_r = \min\{\tau_j | \tau_j \in \mathcal{T}_{out}(v, \mathcal{G}_q) \wedge \tau < \tau_j \leq \tau_e\}$, if $TCV_{\tau_l}(s,u) \cap TCV_{\tau_r}(v,t) \neq \emptyset$, then for all $\tau_b \leq \tau_i < \tau_l$ and $\tau_r < \tau_j \leq \tau_e$, $TCV_{\tau_i}(s,u) \cap TCV_{\tau_j}(v,t) \neq \emptyset$.*

With Lemma 8, for each $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$ with $u \neq s$, $v \neq t$, we only need to perform the intersection operation once to determine if it should be included in $\mathcal{G}_t$. The detailed process of generating $\mathcal{G}_t$ is illustrated in Algorithm 5. Note that, edges are processed in a non-descending temporal order, and we utilize pointers in $TCV.(s,\cdot)$ and $TCV.(\cdot,t)$ to achieve

constant-time access to entries in a similar way as Algorithm 4. A running example of the process is provided as follows.

***Example* 8** *Given $\mathcal{G}_q$ in Fig. 3(c), the precomputed $TCV_{\cdot}(s,\cdot)$ and $TCV_{\cdot}(\cdot,t)$ in Figs. 4(a)-4(b). The resulting tight upper-bound graph $\mathcal{G}_t$ is shown in Fig. 4(c). Take $e(c,f,4)$ as an example, it is contained in $\mathcal{G}_t$ as $TCV_3(s,c) \cap TCV_5(f,t) = \emptyset$.*

***Lemma* 9** *An edge $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_q)$ is in $\mathcal{E}(\mathcal{G}_t)$ iff one of the following two conditions is satisfied:*

i) $u \neq s$, $v \neq t$, $TCV_{\tau_l}(s,u) \cap TCV_{\tau_r}(v,t) = \emptyset$, *where* $\tau_l = \max\{\tau_i | \tau_i \in \mathcal{T}_{in}(u,\mathcal{G}_q) \wedge \tau_b \leq \tau_i < \tau\}$ *and* $\tau_r = \min\{\tau_j | \tau_j \in \mathcal{T}_{out}(v,\mathcal{G}_q) \wedge \tau < \tau_j \leq \tau_e\}$.

ii) $u = s$ *or* $v = t$.

***Theorem* 4** *The time complexity of Algorithm 5 is $O(n+\theta \cdot m)$, and its space complexity is $O(n+m)$.*

V. Escaped Edges Verification

In this section, we propose an Escaped Edges Verification (EEV) method which strictly imposes the simple path constraint on $\mathcal{G}_t$ and generates the exact $tspG$. The main idea of EEV is that we iteratively select an unverified edge $e \in \mathcal{G}_t$ and identify a $p^*_{[\tau_b,\tau_e]}(s,t)$ through e by applying the bidirectional DFS. All the edges along this path belong to $tspG$, and we add them to the result. The process stops until all the edges in $\mathcal{G}_t$ have been verified. Before discussing the details of our method, we first introduce the following ideas.

***Lemma* 10** *Given $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_t)$ with $u \neq s$, $v \neq t$, if i) $\exists\ e(s,u,\tau') \in \mathcal{E}(\mathcal{G}_t)$ s.t. $\tau_b \leq \tau' < \tau$, or ii) $\exists\ e(v,t,\tau') \in \mathcal{E}(\mathcal{G}_t)$ s.t. $\tau < \tau' \leq \tau_e$, then there is a temporal simple path $p^*_{[\tau_b,\tau_e]}(s,t)$ containing the edge $e(u,v,\tau)$.*

By applying Lemmas 2 and 10, we can confirm certain edges belong to $tspG$ without searching for the exact paths containing them, which reduces the number of edges to verify. Moreover, we can accelerate the edge verification process by adding edges across a batch of paths simultaneously based on an identified temporal simple path using the following lemma.

***Lemma* 11** *Given a temporal simple path $p^* = \langle e(u_0,u_1,\tau_1), \ldots, e(u_{l-1},u_l,\tau_l)\rangle$, for all integers $1 \leq i \leq l$, $e(u_{i-1},u_i,\tau_i) \in \mathcal{E}(p^*)$ can be replaced by any $e(u_{i-1},u_i,\tau)$ s.t. i) if $i = 1$, $\tau \in [\tau_b,\tau_{i+1})$, ii) if $1 < i < l-1$, $\tau \in (\tau_{i-1},\tau_{i+1})$, iii) if $i = l$, $\tau \in (\tau_i - 1, \tau_b]$, and the resulting path remains a temporal simple path from s to t within the time interval $[\tau_b,\tau_e]$.*

Following Lemma 11, when a path $p^*_{[\tau_b,\tau_e]}(s,t)$ is identified during the bidirectional DFS, in addition to all the edges on this path itself, edges that are able to replace each of them to form another temporal simple path from s to t within $[\tau_b,\tau_e]$ can be confirmed at the same time. Based on the above analysis, we give the pseudocode of our Escaped Edges Verification method for producing exact $tspG$ in Algorithm 6.

Escaped Edges Verification. In Algorithm 6, Lines 2-5 initialize the verification status for each edge in $\mathcal{G}_t$. For each out-neighbor (u,τ') of s (resp. in-neighbor of t), we mark the edge $e(s,u,\tau')$ (resp. $e(u,t,\tau')$) as verified and add it to $tspG$

Algorithm 6: Escaped Edges Verification

Input : tight upper-bound graph $\mathcal{G}_t$ of a temporal graph $\mathcal{G} = (\mathcal{V},\mathcal{E})$, a source vertex $s \in \mathcal{V}$, a target vertex $t \in \mathcal{V}$ and a time interval $[\tau_b,\tau_e]$

Output: temporal simple path graph $tspG = (\mathcal{V}^*,\mathcal{E}^*)$

1 $\mathcal{V}^* = \emptyset, \mathcal{E}^* = \emptyset$;
2 **for each** $e \in \mathcal{E}(\mathcal{G}_t)$ mark e as unverified;
3 **for each** $e(u,v,\tau)$ *satisfies Lemma 2 or 10* **do**
4 mark $e(u,v,\tau)$ as verified;
5 $\mathcal{V}^* \leftarrow \mathcal{V}^* \cup \{u,v\}$; $\mathcal{E}^* \leftarrow \mathcal{E}^* \cup \{e(u,v,\tau)\}$;
6 **for each** $e(u,v,\tau) \in \mathcal{E}(\mathcal{G}_t)$ **do**
7 **if** $e(u,v,\tau)$ *is verified* **then continue**;
8 $S_v \leftarrow$ an empty stack, $S_e \leftarrow$ an empty stack;
9 **if** $\neg BiDirSearch(e(u,v,\tau))$ **then continue**;
10 build path p^* from S_e;
11 **for** $3 \leq i \leq l(p^*) - 2$ **do**
12 $e(u_{i-2},u_{i-1},\tau_{i-1}) \leftarrow$ the (i-1)-th edge in p^*;
13 $e(u_{i-1},u_i,\tau_i) \leftarrow$ the i-th edge in p^*;
14 $e(u_i,u_{i+1},\tau_{i+1}) \leftarrow$ the (i+1)-th edge in p^*;
15 $\mathcal{V}^* \leftarrow \mathcal{V}^* \cup \{u_{i-1},u_i\}$;
16 **for each** $(w_j,\tau_j) \in N_{out}(u_{i-1},\mathcal{G}_t)$ **do**
17 **if** $w_j = u_i$ *and* $\tau_j \in (\tau_{i-1},\tau_{i+1})$ **then**
18 mark $e(u_{i-1},u_i,\tau_j)$ as verified;
19 $\mathcal{E}^* \leftarrow \mathcal{E}^* \cup \{e(u_{i-1},u_i,\tau_j)\}$;
/* Lemma 11 */
20 **return** $(\mathcal{V}^*,\mathcal{E}^*)$;

based on Lemma 2; we also check the out-neighbors (resp. in-neighbors) of u, if a neighbor (v,τ) with a timestamp $\tau > \tau'$ (resp. $\tau < \tau'$) is found, we mark $e(u,v,\tau)$ (resp. $e(v,u,\tau)$) as verified and add it to $tspG$ according to Lemma 10. In $\mathcal{G}_t$, edges are stored in a non-descending temporal order, Lines 6-7 iterate through those edges and select an unverified one to start a bidirectional DFS for a temporal simple path. From Line 8 to Line 10, global stacks S_v, S_e are maintained throughout the bidirectional DFS. When a temporal simple path is found, we can build it from S_e. The edge confirmation process is illustrated in Lines 11-19. For each edge in this identified path, we confirm it together with all edges that can replace it to form another temporal simple path, add those edges to $tspG$, and mark them as verified. Note that, we omit the confirmation for i-th edge in the path when $i = 1, 2, l(p^*) - 1, l(p^*)$ since it clearly satisfies Lemma 2 or 10 and has been verified in Line 4.

Optimized Bidirectional DFS. To further speed up the path identification process, we propose two optimization techniques utilized in our implementation of bidirectional DFS.

i) **Prioritization of Search Direction**. When an unverified edge $e(u,v,\tau)$ is selected, the bidirectional DFS for identifying a path through $e(u,v,\tau)$ includes a forward search for a forward path $p^*_{[\tau,\tau_e]}(u,t)$ and a backward search for a backward path $p^*_{[\tau_b,\tau]}(s,v)$. Assume $\tau - \tau_b > \tau_e - \tau$, based on Remark 1, the backward path is potentially longer than the forward path. If we perform the backward search prior

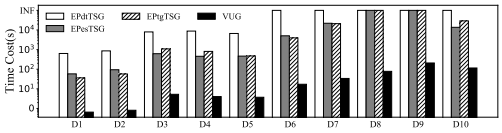


Fig. 5: Response time on all the datasets

to the forward search, we may not find a valid forward path since the vertices have been possessed by the longer backward path, and the DFS will need to backtrack extensively due to the failure to form a temporal simple path. Following this idea, if $\tau - \tau_b > \tau_e - \tau$, we perform the forward search followed by the backward search; otherwise, the reverse.

ii) **Neighbor Exploration Order**. Intuitively, a shorter temporal path has a lower probability of revisiting a vertex than a longer one. To guide the DFS in favor of shorter paths, at each vertex, we traverse its neighbors following a temporal order. Specifically, in the forward search, we explore out-neighbors in a non-ascending temporal order; while in the backward search, we explore in-neighbors in a non-descending temporal order.

Note that, due to the space limitation, the pseudocode of BiDirSearch can be found in Appendix B online [35].

***Theorem* 5** *Algorithm 6 needs $O(m \cdot d_t^{\theta-1})$ time, where d_t is the largest degree of the vertices in $\mathcal{G}_t$, i.e., $d_t = \max_{u \in \mathcal{V}(\mathcal{G}_t)}\{\max(|N_{in}(u, \mathcal{G}_t)|, |N_{out}(u, \mathcal{G}_t)|)\}$ and $\theta = \tau_e - \tau_b + 1$. Its space complexity is $O(n + m)$.*

VI. Experiments

A. Experiment Setup

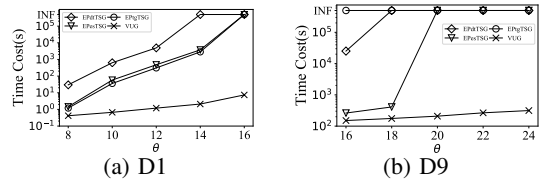
Algorithms. To the best of our knowledge, we are the first to investigate the temporal simple path graph generation problem. Therefore, we implement and evaluate the following four algorithms in our experiments. Three baseline methods i.e., `EPdtTSG`, `EPesTSG`, `EPtgTSG`, whose details can be found in Section III-A. `VUG`: Algorithm 1 proposed in Section III-B with all the optimized strategies.

Datasets. We employ 10 real-world temporal graphs in our experiments. Details of these datasets are summarized in TABLE I of Appendix C online [35].

Parameters and workloads. For each dataset, we generate 1000 random queries with different source vertices s, target vertices t, time intervals $[\tau_b, \tau_e]$ where s can temporally reach t within $[\tau_b, \tau_e]$ and report the total query time on each dataset. We conduct the experiment by varying $\tau_e - \tau_b + 1$ (i.e., θ), whose default values are set following existing works about path problems in temporal graphs (e.g., [12], [15], [36]). Details are shown in the last column of TABLE I. For those algorithms that cannot finish within 12 hours, we set them as **INF**. All the programs are implemented in standard C++. All the experiments are performed on a server with Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz and 256G RAM.

B. Performance Comparison with Baselines

Exp-1: Response time on all the datasets. Fig. 5 reports the total response time of `EPdtTSG`, `EPesTSG`, `EPtgTSG` and `VUG` for 1000 queries over all the datasets under the default settings. As observed, `EPesTSG` and `EPtgTSG` are faster than `EPdtTSG` due to their optimization for tighter upper-bound graphs, i.e., integrating the temporal path constraint. `VUG` outperforms the other three algorithms on all the datasets by a significant margin as a result of *i*) it further excludes many unpromising edges by incorporating the simple path constraint when generating the tight upper-bound graph; and *ii*) it avoids lots of verifications for the existence of edges in the final result. For example, on the datasets D8 and D9, three baseline methods cannot finish queries within 12 hours due to the large search space, while `VUG` only takes 78 seconds and 208 seconds to return the result, respectively.



(a) D1 (b) D9

Fig. 6: Response time by varying parameter θ

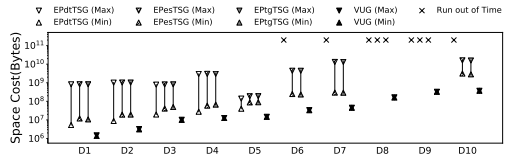


Fig. 7: Space consumption comparison

Exp-2: Response time by varying parameter θ. In Fig. 6, we report the total response time of `EPdtTSG`, `EPesTSG`, `EPtgTSG` and `VUG` for 1000 queries by varying θ on the datasets D1 and D9. Due to the space limitation, results on the other datasets can be found in Appendix C online [35]. We can see that the response time of the three baselines grows exponentially as θ increases, while the response time of `VUG` grows modestly, demonstrating its better scalability. For example, when θ varies from 8 to 12 on D1, the total response time of `EPdtTSG`, `EPesTSG`, `EPtgTSG` increases by a factor of 165, 320, 259, respectively, while that of `VUG` only increases by a factor of 3.

Exp-3: Space consumption comparison. Fig. 7 reports the space consumption of `VUG` and the three baseline algorithms. For each algorithm, we report its maximum and minimum space consumption among 1000 queries on each dataset under the default settings. As observed, `VUG` consistently consumes less space compared to baseline algorithms, attributed to the fact that it avoids enumerating and explicitly storing all temporal simple paths. In addtion, `VUG` maintains a stable space consumption across different queries, regardless of the changing number of temporal simple paths, whereas the baselines exhibit exponential differences between their maximum and minimum space consumption. This supports our theoretical analysis in Theorems 1-5, that is, the space complexity of `VUG` is linear in the number of vertices and edges in the datasets.

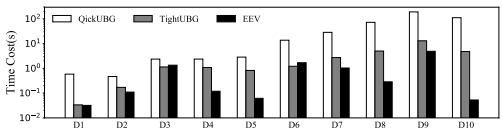


Fig. 8: Response time of each phase in VUG

C. Performance Evaluation of Each Phase in VUG

Exp-4: Response time of each phase in VUG. In this experiment, we test the total response time of each phase in VUG (i.e., QuickUBG, TightUBG and EEV) processing 1000 queries on each dataset. The results are illustrated in Fig. 8. As we can see, the response time of EEV is the shortest among those three phases in most datasets. For example, on D10, the running time of EEV accounts for only 0.04% of the total response time, indicating that the bidirectional DFS, though theoretically exponential, has a limited practical overhead.

Exp-5: Evaluation of upper-bound graph generation. In this experiment, we evaluate the performance of five upper-bound graph generation methods, i.e., dtTSG, esTSG, tgTSG, QuickUBG and TightUBG. For each algorithm, we define the upper-bound ratio as the percentage of the resulting $tspG$ in the corresponding upper-bound graph, i.e., $\frac{\text{the number of edges in the result graph } tspG}{\text{the number of edges in the upper-bound graph obtained by the algorithm}}$, and then calculate it for 1000 queries on each dataset, whose average upper-bound ratio is reported in TABLE II. As observed, the upper-bound ratios achieved by dtTSG are consistently less than 0.1%. The upper-bound graphs produced by esTSG and tgTSG are comparatively tighter, and VUG generates the tightest upper-bound graph, achieving an upper-bound ratio exceeding 90% on 8 datasets.

As discussed in Section IV-A, while tgTSG and QuickUBG achieve the same effect in terms of graph reduction, QuickUBG reduces the time complexity of tgTSG by a factor of $O(\log n)$. To further demonstrate the efficiency of QuickUBG, we report the total response time of tgTSG and QuickUBG for 1000 queries on each dataset. Details are shown in Fig. 9. QuickUBG is more efficient since it further optimizes the traversal process. For example, on the dataset D7, tgTSG takes 2.4 hours to complete the upper-bound graph generation, while QuickUBG only takes 32 seconds.

We also evaluate the impact of θ on upper-bound graph generation. Fig. 10 reports the response time and the upper-bound ratio for QuickUBG and TightUBG on the two largest datasets D9 and D10 (results on other datasets can be found in Appendix C online [35]). As shown, the upper-bound ratio slightly decreases with increasing θ in most cases due to the introduction of more redundant edges that can not be pruned. For example, on D9, when θ increases from 16 to 24, the ratio decreases from 98.5% to 91.6%. However, in some cases, such as θ from 23 to 25 on D10, the ratio slightly increases from 88.2% to 99.6%, because certain previously redundant edges can be included in newly established temporal simple paths from s to t within a larger time interval and thus belong to $tspG$. Meanwhile, the overhead of QuickUBG and TightUBG remains stable across various settings of θ, which

TABLE II: The average upper-bound ratio (%)

		D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
dtTSG		<0.1	<0.1	<0.1	<0.1	<0.1	-	-	-	-	-
esTSG		10.4	32.1	<0.1	0.2	13.5	9.8	15.8	-	-	64.6
tgTSG		59.4	51.1	3.4	4.6	30.6	24.6	28.4	-	-	90.9
VUG-UB	QuickUBG	59.4	51.1	3.4	4.6	30.6	24.6	28.4	88.9	38.9	90.9
	TightUBG	94.9	98.4	70.6	90.1	97.2	92.4	87.9	98.8	95.5	99.6

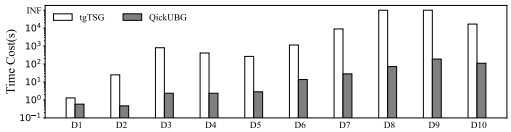


Fig. 9: Response time of tgTSG and QuickUBG

confirms the scalability. For example, on the dataset D9, when θ increases from 16 to 24, the time overhead of VUG increases from 151 seconds to 315 seconds. Within this, the overhead of QuickUBG ranges from 140 seconds to 215 seconds, and that of TightUBG ranges from 9 seconds to 14 seconds.

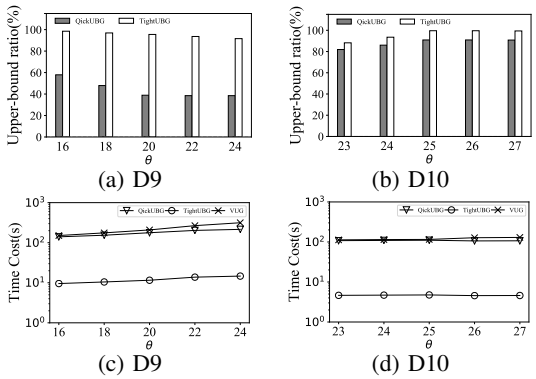
Exp-6: Evaluation of EEV. We compare EEV with path enumeration method by applying them separately on $\mathcal{G}_t$ and generating $tspG$. The total response time for 1000 queries on D1 and D8 is reported in Fig. 11 (results on the other datasets can be found in Appendix C online [35]). As observed, EEV accelerates the process of generating $tspG$ by at least an order of magnitude compared to the enumeration method, since based on a tight upper-bound graph, EEV has the potential to reduce repeated searches as opposed to the path enumeration method, thereby accelerating the generation of $tspG$. For example, when θ is 12 on D8, the enumeration-based method takes 796 seconds, while EEV only takes 44 seconds.

D. Effectiveness Evaluation of tspG

Exp-7: Number of edges/paths in $tspG$. In this experiment, we report the number of edges in $tspG$ and the number of temporal simple paths it contains on D1 and D8 by varying θ. Results are shown in Fig. 12 (results on all other datasets can be found in Appendix C online [35]). As we can see, the number of temporal simple paths in $tspG$ far exceeds the number of edges in $tspG$. For example, when $\theta = 10$ on D1, the generated $tspG$ with 659 edges contains more than 3 million temporal simple paths. This means many edges are shared among multiple paths, highlights the effectiveness of representing all paths as a single graph structure.

Exp-8: Case study on SFMTA. To demonstrate the effectiveness of our model, we collect the GTFS transit data from the San Francisco Municipal Transportation Agency (SFMTA[1]), which includes 936,188 scheduled trips across 3,267 stops made by 25,533 buses in a single day. We model this data as a temporal directed graph where each edge (u, v) is associated with a timestamp that records the departure time of a bus from a stop u to the next stop v [23]. Given the query ("Silver Ave", "30th St", [9:20, 9:30]), Fig. 13 illustrates the corresponding temporal simple path graph $tspG$ with 8 transit stops (i.e.,

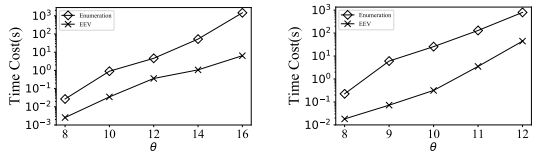
[1]https://www.sfmta.com/reports/gtfs-transit-data



(a) D9 (b) D10

(c) D9 (d) D10

Fig. 10: Evaluation of upper-bound graph generation



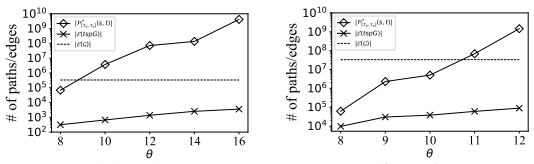
(a) D1 (b) D8

Fig. 11: Evaluation of EEV

vertices) and 17 transit trips (i.e., edges). This graph provides possible transfer options that allow passengers to reach the destination within the given time interval. For example, if passengers miss buses departing from "Silver Ave" before 9:23, they can still board Bus 469 at 9:24, transfer to Bus 291 at "Richland Ave", and ultimately arrive at "30th St" on time. This demonstrates the utility of $tspG$ in capturing dynamic transit options and aiding real-time travel planning.

VII. Related Work

Simple Path Enumeration. Graphs are widely used to model relationships between entities across various domains [37]–[41]. Simple path enumeration is one of the fundamental problems in graph analysis and has been extensively studied in [1]–[13]. Peng et al. [1] propose prune-based algorithms that dynamically maintain hop bounds for each vertex, avoiding the exploration of unpromising branches when the remaining hop budget is smaller than a vertex's hop bound. Sun et al. [2] introduce index-based algorithms to reduce the search space towards real-time enumeration queries. Yuan et al. [3] focus on batch hop-constrained *s-t* path enumeration, sharing common computations across queries to boost efficiency. [4] and [5] further study the implementation on FPGA and distributed environments, respectively. In addition, Li et al. [6] study labelled hop-constrained *s-t* simple path enumeration on billion-scale labelled graphs. Rizzi et al. [11] propose length-constrained *s-t* simple path enumeration on non-negative weighted graphs, where the total weight of each path does not exceed α. Jin et al. [12] explore multi-constrained *s-t* path enumeration on temporal graphs. Mutzel et al. [13] introduce two bicriteria temporal min-cost path problems, focusing on enumerating simple paths that are efficient w.r.t. costs and duration or costs and arrival times. Although there are many works on the path enumeration, it is still inefficient to extend the existing solutions to our problem.



(a) D1 (b) D8

Fig. 12: Numbers of paths/edges in $tspG$

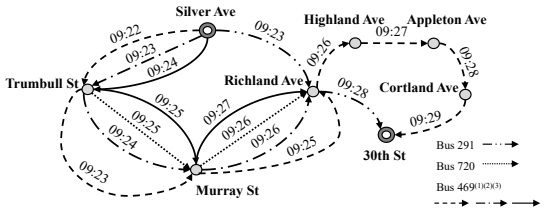


Fig. 13: Case study on SFMTA

Path Graph Queries. Path graph captures the structural relationships between vertices by characterizing connections through reachable paths. Existing works [28]–[30] address path graph queries between two given vertices s and t. Liu et al. [29] define the k-hop *s-t* subgraph query, which involves all paths from s to t within k-hops. Cai et al. [30] extend this problem and further study hop-constraint *s-t* simple path graphs. Wang et al. [28] formalize the *s-t* shortest path graph, containing all shortest paths between s and t. However, these graphs lack temporal information, and a naive extension of their methods to our problem is inefficient. Some complex path-based graph queries do involve temporal information [34], [42], [43]. Bhadra et al. [43] compute strongly connected components where any pair of vertices within a component are connected by a temporal path, which might be too rigid and inadequate to identify relationships in many real-life situations. Cheng et al. [42] generate the associated influence digraph where each edge represents a non-decreasing path in the original graph. Bast et al. [34] generate a graph that contains a set of optimal connections with multi-criteria shortest paths. However, the resulting graphs generated in these works are not well-suited for illustrating the underlying connections, as they are not subgraphs of the original graph.

VIII. Conclusion

Temporal graph is an important structure for modeling many real-world applications. To analyze the relationships between vertices in temporal graphs, in this paper, we study the problem of generating temporal simple path graph. To solve the problem, three reasonable baselines are first proposed by explicitly enumerating all temporal simple paths. To overcome the limitations in baselines and scale for larger networks, we further propose an efficient algorithm VUG with two main components, i.e., Upper-bound Graph Generation and Escaped Edges Verification. Extensive experiments on 10 real-world temporal graphs show that VUG significantly outperforms all baselines by at least two orders of magnitude.

Acknowledgments. This work was supported by UoW R6288 and ARC DP240101322, DP230101445.

REFERENCES

[1] Y. Peng, X. Lin, Y. Zhang, W. Zhang, L. Qin, and J. Zhou, "Efficient hop-constrained s-t simple path enumeration," *VLDB J.*, vol. 30, no. 5, pp. 799–823, 2021.

[2] S. Sun, Y. Chen, B. He, and B. Hooi, "Pathenum: Towards real-time hop-constrained s-t path enumeration," in *SIGMOD*, 2021, pp. 1758–1770.

[3] L. Yuan, K. Hao, X. Lin, and W. Zhang, "Batch hop-constrained s-t simple path query processing in large graphs," in *ICDE*, 2024, pp. 2557–2569.

[4] Z. Lai, Y. Peng, S. Yang, X. Lin, and W. Zhang, "PEFP: efficient k-hop constrained s-t simple path enumeration on FPGA," in *ICDE*, 2021, pp. 1320–1331.

[5] K. Hao, L. Yuan, and W. Zhang, "Distributed hop-constrained s-t simple path enumeration at billion scale," *Proc. VLDB Endow.*, vol. 15, no. 2, pp. 169–182, 2021.

[6] X. Li, K. Hao, Z. Yang, X. Cao, W. Zhang, L. Yuan, and X. Lin, "Hop-constrained s-t simple path enumeration in billion-scale labelled graphs," in *WISE*, 2022, pp. 49–64.

[7] J. Zhang, S. Yang, D. Ouyang, F. Zhang, X. Lin, and L. Yuan, "Hop-constrained s-t simple path enumeration on large dynamic graphs," in *ICDE*, 2023, pp. 762–775.

[8] Q. Liang, D. Ouyang, F. Zhang, J. Yang, X. Lin, and Z. Tian, "Efficient regular simple path queries under transitive restricted expressions," *Proc. VLDB Endow.*, vol. 17, no. 7, pp. 1710–1722, 2024.

[9] E. Birmelé, R. A. Ferreira, R. Grossi, A. Marino, N. Pisanti, R. Rizzi, and G. Sacomoto, "Optimal listing of cycles and st-paths in undirected graphs," in *SODA*, 2013, pp. 1884–1896.

[10] R. Grossi, A. Marino, and L. Versari, "Efficient algorithms for listing k disjoint st-paths in graphs," in *LATIN*, 2018, pp. 544–557.

[11] R. Rizzi, G. Sacomoto, and M. Sagot, "Efficiently listing bounded length st-paths," in *IWOCA*, 2014, pp. 318–329.

[12] Y. Jin, Z. Chen, and W. Liu, "Enumerating all multi-constrained s-t paths on temporal graph," *Knowl. Inf. Syst.*, vol. 66, no. 2, pp. 1135–1165, 2024.

[13] P. Mutzel and L. Oettershagen, "On the enumeration of bicriteria temporal paths," in *TAMC*, 2019, pp. 518–535.

[14] X. Chen, K. Wang, X. Lin, W. Zhang, L. Qin, and Y. Zhang, "Efficiently answering reachability and path queries on temporal bipartite graphs," *Proc. VLDB Endow.*, vol. 14, no. 10, pp. 1845–1858, 2021.

[15] X. Gou, X. Ye, L. Zou, and J. X. Yu, "LM-SRPQ: efficiently answering regular path query in streaming graphs," *Proc. VLDB Endow.*, vol. 17, no. 5, pp. 1047–1059, 2024.

[16] Y. Wu, R. Sun, X. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Efficient maximal temporal plex enumeration," in *ICDE*, 2024, pp. 3098–3110.

[17] X. Cai, X. Ke, K. Wang, L. Chen, T. Zhang, Q. Liu, and Y. Gao, "Efficient temporal butterfly counting and enumeration on temporal bipartite graphs," *Proc. VLDB Endow.*, vol. 17, no. 4, pp. 657–670, 2023.

[18] K. Chen, D. Wen, W. Zhang, Y. Zhang, X. Wang, and X. Lin, "Querying structural diversity in streaming graphs," *Proc. VLDB Endow.*, vol. 17, no. 5, pp. 1034–1046, 2024.

[19] D. Wen, Y. Huang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Efficiently answering span-reachability queries in large temporal graphs," in *ICDE*, 2020, pp. 1153–1164.

[20] H. Wu, Y. Huang, J. Cheng, J. Li, and Y. Ke, "Reachability and time-based path queries in temporal graphs," in *ICDE*, 2016, pp. 145–156.

[21] T. Zhang, Y. Gao, L. Chen, W. Guo, S. Pu, B. Zheng, and C. S. Jensen, "Efficient distributed reachability querying of massive temporal graphs," *VLDB J.*, vol. 28, no. 6, pp. 871–896, 2019.

[22] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, and Y. Xu, "Path problems in temporal graphs," *Proc. VLDB Endow.*, vol. 7, no. 9, pp. 721–732, 2014.

[23] S. Wang, W. Lin, Y. Yang, X. Xiao, and S. Zhou, "Efficient route planning on public transportation networks: A labelling approach," in *SIGMOD*, 2015, pp. 967–982.

[24] Y. Wu, R. Sun, X. Wang, D. Wen, Y. Zhang, L. Qin, and X. Lin, "Efficient maximal frequent group enumeration in temporal bipartite graphs," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3243–3255, 2024.

[25] Z. Gao, C. Cheng, Y. Yu, L. Cao, C. Huang, and J. Dong, "Scalable motif counting for large-scale temporal graphs," in *ICDE*, 2022, pp. 2656–2668.

[26] S. Gurukar, S. Ranu, and B. Ravindran, "COMMIT: A scalable approach to mining communication motifs from dynamic networks," in *SIGMOD*, 2015, pp. 475–489.

[27] Y. Gao, T. Zhang, L. Qiu, Q. Linghu, and G. Chen, "Time-respecting flow graph pattern matching on temporal graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 10, pp. 3453–3467, 2021.

[28] Y. Wang, Q. Wang, H. Koehler, and Y. Lin, "Query-by-sketch: Scaling shortest path graph queries on very large networks," in *SIGMOD*, 2021, pp. 1946–1958.

[29] Y. Liu, Q. Ge, Y. Pang, and L. Zou, "Hop-constrained subgraph query and summarization on large graphs," in *DASFAA*, 2021, pp. 123–139.

[30] Y. Cai, S. Liu, W. Zheng, and X. Lin, "Towards generating hop-constrained s-t simple path graphs," *SIGMOD*, vol. 1, no. 1, pp. 61:1–61:26, 2023.

[31] A. Casteigts, T. Corsini, and W. Sarkar, "Simple, strict, proper, happy: A study of reachability in temporal graphs," *Theor. Comput. Sci.*, vol. 991, p. 114434, 2024.

[32] S. Eubank, H. Guclu, V. Anil Kumar, M. V. Marathe, A. Srinivasan, Z. Toroczkai, and N. Wang, "Modelling disease outbreaks in realistic urban social networks," *Nature*, vol. 429, no. 6988, pp. 180–184, 2004.

[33] X. Qiu, W. Cen, Z. Qian, Y. Peng, Y. Zhang, X. Lin, and J. Zhou, "Real-time constrained cycle detection in large dynamic graphs," *Proc. VLDB Endow.*, vol. 11, no. 12, pp. 1876–1888, 2018.

[34] H. Bast, E. Carlsson, A. Eigenwillig, R. Geisberger, C. Harrelson, V. Raychev, and F. Viger, "Fast routing in very large public transportation networks using transfer patterns," in *ESA*, 2010, pp. 290–301.

[35] https://github.com/zhiyangtang/VUG-for-TSPG.

[36] L. Li, W. Hua, X. Du, and X. Zhou, "Minimal on-road time route scheduling on time-dependent graphs," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1274–1285, 2017.

[37] J. Wang, Y. Wu, X. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "Efficient influence minimization via node blocking," *Proc. VLDB Endow.*, vol. 17, no. 10, pp. 2501–2513, 2024.

[38] R. Sun, Y. Wu, X. Wang, C. Chen, W. Zhang, and X. Lin, "Efficient balanced signed biclique search in signed bipartite graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 3, pp. 1069–1083, 2024.

[39] J. Wang, Y. Wu, X. Wang, C. Chen, Y. Zhang, and L. Qin, "Effective influence maximization with priority," in *THE WEB CONFERENCE 2025*.

[40] Y. Wu, J. Wang, R. Sun, C. Chen, X. Wang, and Y. Zhang, "Targeted filter bubbles mitigating via edges insertion," in *WWW (Companion Volume)*. ACM, 2024, pp. 734–737.

[41] R. Sun, Y. Wu, C. Chen, X. Wang, W. Zhang, and X. Lin, "Maximal balanced signed biclique enumeration in signed bipartite graphs," in *ICDE*. IEEE, 2022, pp. 1887–1899.

[42] E. Cheng, J. W. Grossman, and M. J. Lipman, "Time-stamped graphs and their associated influence digraphs," *Discret. Appl. Math.*, vol. 128, no. 2-3, pp. 317–335, 2003.

[43] S. Bhadra and A. Ferreira, "Complexity of connected components in evolving graphs and the computation of multicast trees in dynamic networks," in *ADHOC-NOW*, 2003, pp. 259–270.